working_with_numba

October 12, 2024

1 Working with Numba

1.1 Numba

Numba is an accelerator library for Python, which just-in time compiles Python code into fast machine code. - If used right, its performance can be close to optimized C code. - Moreover, it supports offloading of kernels to GPU devices and shared memory parallelism.

1.2 Just in Time

Describes compiling code *during* excecution of the program rather than before.

Why do we do this?

- We can write code once and target multiple platforms
  - Javascript and Java exploit this
- The specific code can be optimized for the problem given inputs and CPU architecture

Of course when code is first run, there will be a delay as it is compiled, so keep that in mind

- `%timeit` considers this when used

1.3 Example

The following example from the Numba homepage provides a very first idea of what Numba does.

[1]:
```
from numba import njit, jit
from numpy import arange

# jit decorator tells Numba to compile this function.
# The argument types will be inferred by Numba when function is called.
@jit
def sum2d(arr):
    M, N = arr.shape
    result = 0.0
    for i in range(M):
        for j in range(N):
            result += arr[i,j]
    return result
```

```
a = arange(9).reshape(3,3)
print(f'Sum2D: \t\t{sum2d(a)}')
print(f'Numpy sum: \t{a.sum()}')
```

```
Sum2D:          36.0
Numpy sum:      36
```

- On its first call the `sum2d` function is just-in-time compiled into fast executable code and then executed.
- All that is needed is the decorator `@njit`.

1.4 `jit` and `njit`

- `jit` is the most general compilation mode
  - Compile wide range of Python code
    - `object` mode invoked if needed
  - Custom Python objects are supported
  - May require jumping into and out of Python interpretor in `object` mode
- `njit` is a restricted mode
  - Basically `jit(nopython=True)`
  - Enforces that Python interpretor is not allowed
  - Generally generates fastest code
  - Will refuse to compile if condition is not met
  - Most likely fastest code

If the code doesn't require Python then `jit` and `njit` can behave the same! The difference is whether to compile the code or not.

[2]:
```
from numba import jit, njit
import pandas as pd

x = {'a': [1, 2, 3], 'b': [20, 30, 40]}


def use_pandas(a):
    df = pd.DataFrame.from_dict(a)
    df += 1
    return df.cov()

@jit(forceobj=True) # Need to use object mode, try and compile loops!
def use_pandas_jit_obj(a): # Function will not benefit from Numba jit
    df = pd.DataFrame.from_dict(a) # Numba doesn't know about pd.DataFrame
    df += 1                        # Numba doesn't understand what this is
    return df.cov()                # or this!

use_pandas_jit_obj(x)

%timeit use_pandas(x)
```

```
%timeit use_pandas_jit_obj(x)
```

```
581  s ± 18.3  s per loop (mean ± std. dev. of 7 runs, 1,000 loops each)
608  s ± 5.69  s per loop (mean ± std. dev. of 7 runs, 1,000 loops each)
```

Numba also solves the GIL issue we had before using the parameter `nogil`

[3]:
```
import numpy as np
import threading
import multiprocessing

@jit(nopython=True, nogil=True)
def worker_nogil(arr1, arr2, arr3, chunk):
    """The thread worker."""

    for index in chunk:
        arr3[index] = arr1[index] + arr2[index]

def worker(arr1, arr2, arr3, chunk):
    """The thread worker."""

    for index in chunk:
        arr3[index] = arr1[index] + arr2[index]

nthreads = multiprocessing.cpu_count()

n = 1000000
a = np.random.randn(n)
b = np.random.randn(n)

c = np.empty(n, dtype='float64')


def run_with_n_threads(chunks, nthreads, worker):

    all_threads = []
    for chunk in chunks:
        thread = threading.Thread(target=worker, args=(a, b, c, chunk))
        all_threads.append(thread)
        thread.start()

    for thread in all_threads:
        thread.join()
```

Lets test our functions from last week:

[4]:
```
chunks = np.array_split(range(n), 1)
print("With GIL")
%timeit run_with_n_threads(chunks,1,worker)
```

```
print("Without GIL")
%timeit run_with_n_threads(chunks,1, worker_nogil)
```

```
With GIL
670 ms ± 14.4 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)
Without GIL
2.73 ms ± 82.4  s per loop (mean ± std. dev. of 7 runs, 100 loops each)
```

[5]:
```
chunks = np.array_split(range(n), 2)
print("With GIL")
%timeit run_with_n_threads(chunks,2, worker)
print("Without GIL")
%timeit run_with_n_threads(chunks,2, worker_nogil)
```

```
With GIL
644 ms ± 2.11 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)
Without GIL
2.01 ms ± 66.6  s per loop (mean ± std. dev. of 7 runs, 1,000 loops each)
```

We will see later that numba provides an alternative way of exploiting threading.

1.5 Computing Mandelbrot

In the following we want to use Numba to compute the Mandelbrot fractal and measure its performance. First, we define a simple convenient timer in Python.

[6]:
```
import time

class Timer:
    def __enter__(self):
        self.start = time.time()
        return self

    def __exit__(self, *args):
        self.end = time.time()
        self.interval = self.end - self.start
```

We define our quadratic form:

$$z_{n+1} = z_n + c$$

where $z_0 = 0$, $c = x + yj$

- We will iterate z and determine membership in the Mandelbrot when $|z_n| \geq 2$
- c is a complex number determined by pixels (x, y) as $c = x + yi$
- We define a max iteration N_{iter} which stops the iteration at some point
- If max iteration is reached, reject the point (make it black)

[7]:
```
def mandel(x, y, max_iters):
    """
    Given the real and imaginary parts of a complex number,
    determine if it is a candidate for membership in the Mandelbrot
    set given a fixed number of iterations.
    """
    i = 0
    c = complex(x,y)
    z = 0.0j
    for i in range(max_iters):
        z = z*z + c
        if (z.real*z.real + z.imag*z.imag) >= 4:
            return i

    return 255
```

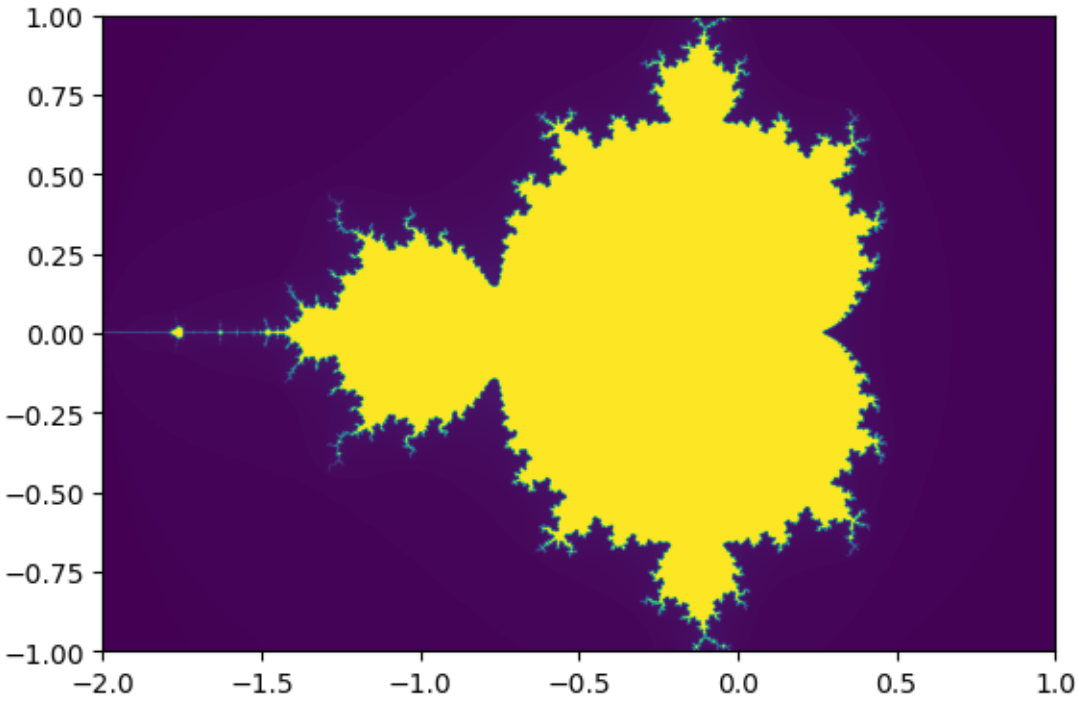
Now we create our fractal given x and y

[8]:
```
def create_fractal(min_x, max_x, min_y, max_y, image, iters):
    height = image.shape[0]
    width = image.shape[1]

    pixel_size_x = (max_x - min_x) / width
    pixel_size_y = (max_y - min_y) / height
    for x in range(width):
        real = min_x + x * pixel_size_x
        for y in range(height):
            imag = min_y + y * pixel_size_y
            color = mandel(real, imag, iters)
            image[y, x] = color

    return image
```

Now we plot it

[9]:
```
#from __future__ import print_function, division, absolute_import
import numpy as np
from pylab import imshow, jet, show, ion

image = np.zeros((2000, 3000), dtype=np.uint8)

with Timer() as t:
    mandelbrot = create_fractal(-2.0, 1.0, -1.0, 1.0, image, 20)
print("Time to create fractal: {0}".format(t.interval))

imshow(mandelbrot, extent=[-2, 1, -1, 1])
show()
```

```
Time to create fractal: 21.004082918167114
```



This is fairly slow. - The problem is that we have three nested for-loops.

- In each inner iteration a call to the Python interpeter needs to be performed.
- Python is not designed for speedy handling of such loop constructs.

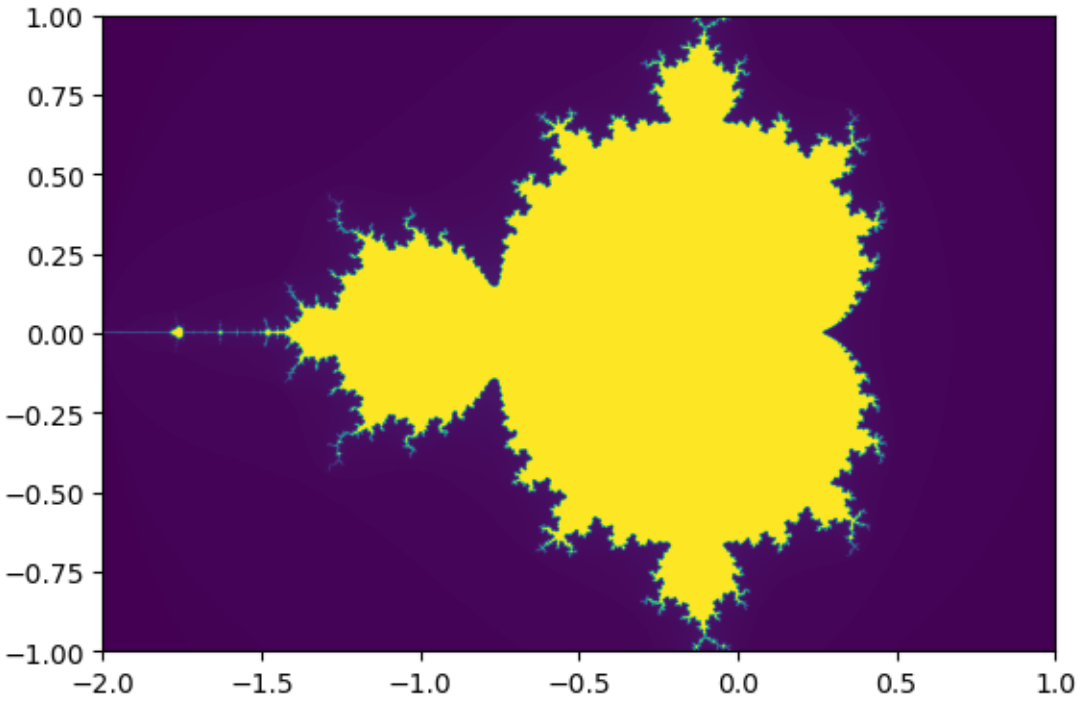
However, we can improve it by enabling Just-In-Time compilation of the routines with Numba.

[10]:
```
from numba import jit
import numpy as np
from pylab import imshow, jet, show, ion

@njit
def mandel(x, y, max_iters):
    """
    Given the real and imaginary parts of a complex number,
    determine if it is a candidate for membership in the Mandelbrot
    set given a fixed number of iterations.
    """
    i = 0
    c = complex(x,y)
    z = 0.0j
    for i in range(max_iters):
        z = z*z + c
        if (z.real*z.real + z.imag*z.imag) >= 4:
            return i
```

```
    return 255

@njit
def create_fractal(min_x, max_x, min_y, max_y, image, iters):
    height = image.shape[0]
    width = image.shape[1]

    pixel_size_x = (max_x - min_x) / width
    pixel_size_y = (max_y - min_y) / height
    for x in range(width):
        real = min_x + x * pixel_size_x
        for y in range(height):
            imag = min_y + y * pixel_size_y
            color = mandel(real, imag, iters)
            image[y, x] = color

    return image

image = np.zeros((2000, 3000), dtype=np.uint8)

with Timer() as t:
    mandelbrot = create_fractal(-2.0, 1.0, -1.0, 1.0, image, 20)
print("Time to create fractal: {0}".format(t.interval))
```

```
Time to create fractal: 0.5231552124023438
```

[11]:
```
imshow(mandelbrot, extent=[-2, 1, -1, 1])
show()
```



1.6 Parallelisation

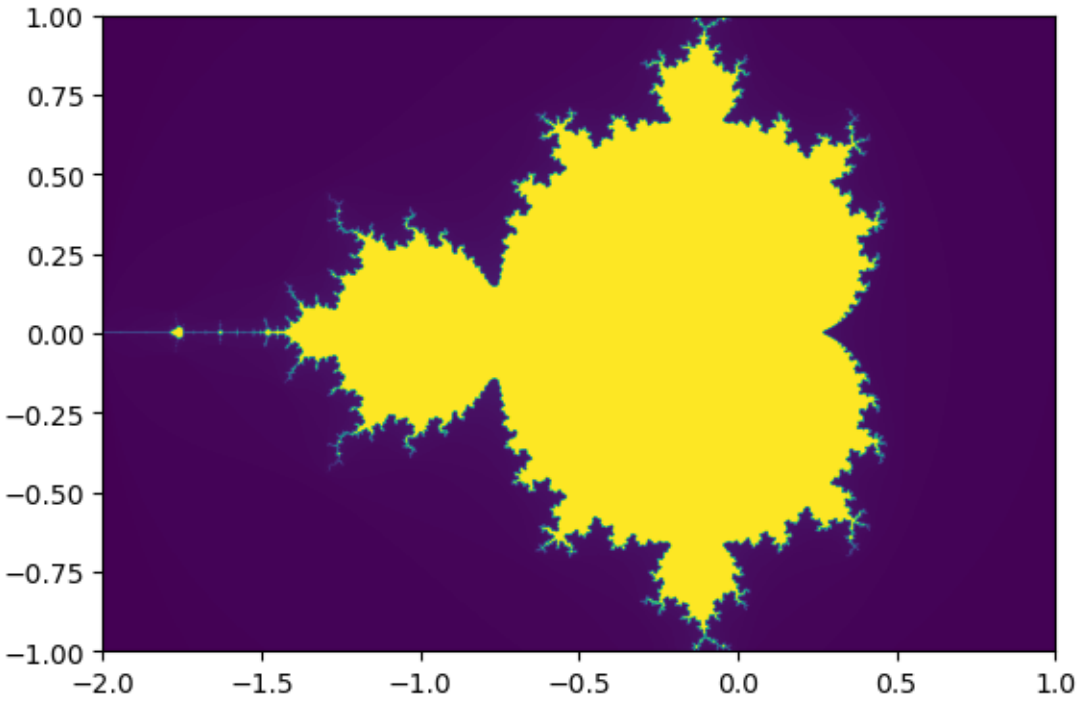
We can go one step further and use multithreading with `prange`

[15]:
```
from numba import njit, prange
import numpy as np
from pylab import imshow, jet, show, ion

@njit
def mandel(x, y, max_iters):
    """
    Given the real and imaginary parts of a complex number,
    determine if it is a candidate for membership in the Mandelbrot
    set given a fixed number of iterations.
    """
    i = 0
    c = complex(x,y)
    z = 0.0j
    for i in range(max_iters):
        z = z*z + c
        if (z.real*z.real + z.imag*z.imag) >= 4:
            return i

    return 255
```

```
@njit(['uint8[:,:](float64, float64, float64, float64, uint8[:, :], uint8)'],␣
 ↪parallel=True)
def create_fractal(min_x, max_x, min_y, max_y, image, iters):
    height = image.shape[0]
    width = image.shape[1]

    pixel_size_x = (max_x - min_x) / width
    pixel_size_y = (max_y - min_y) / height
    for x in prange(width):
        real = min_x + x * pixel_size_x
        for y in range(height):
            imag = min_y + y * pixel_size_y
            color = mandel(real, imag, iters)
            image[y, x] = color
    return image
```

[16]:
```
with Timer() as t:
    mandelbrot = create_fractal(-2.0, 1.0, -1.0, 1.0, image, 20)
print("Time to create fractal: {0}".format(t.interval))

imshow(mandelbrot, extent=[-2, 1, -1, 1])
show()
```

```
Time to create fractal: 0.1741619110107422
```

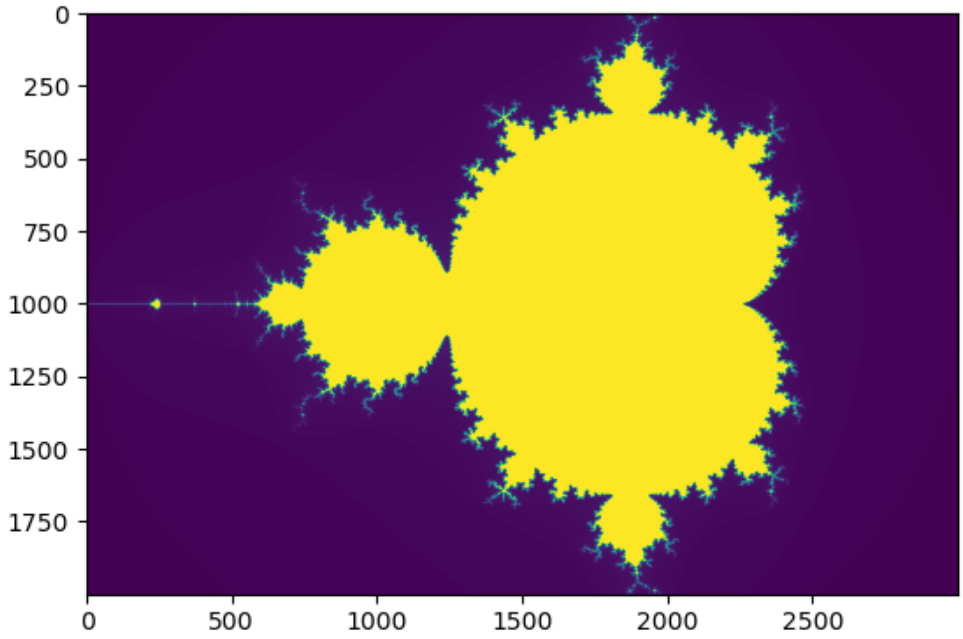


- The key to the parallelization is the `prange` command in the for-loop.
- Tells Numba to spread out the computation to multiple CPU cores.
- However, it is essential that Numba knows all data types, so that no Python calls will be performed during the parallel loop.

We are now working in real time and can do this!

[17]:
```
%matplotlib widget
from ipywidgets import interact
from pylab import subplots
fig, ax = subplots()

image = np.zeros((2000, 3000), dtype=np.uint8)

im = ax.imshow(image)

@interact(x=(-2,2,0.001),y=(-2,2,0.001),zoom=(0.0001, 2,0.0001), niters=(5,80))
def update(x=0,y=0,zoom=1.0, niters=27):
    image[...] = 0.0
    mandelbrot = create_fractal(-2.0*zoom + x, 1.0*zoom + x, -1.0*zoom + y,␣
 ↪1*zoom + y, image, niters)
    im.set_data(mandelbrot)
    im.set_clim(0,255)
```

```
interactive(children=(FloatSlider(value=0.0, description='x', max=2.0, min=-2.0,␣
 ↪step=0.001), FloatSlider(valu…
```



1.7 Code inspection

We can easily inspect the code that Numba generates. Consider the following simple function.

[18]:
```
@njit
def mysum(a, b):
    return a + b

c = mysum(3, 4)
```

We can now inspect the generated LLVM code for this function.

[19]:
```
for v, k in mysum.inspect_llvm().items():
    print(v, k)
```

```
(int64, int64) ; ModuleID = 'mysum'
source_filename = "<string>"
target datalayout =
"e-m:o-p270:32:32-p271:32:32-p272:64:64-i64:64-f80:128-n8:16:32:64-S128"
target triple = "x86_64-apple-darwin21.6.0"

@.const.mysum = internal constant [6 x i8] c"mysum\00"
```

```
@_ZN08NumbaEnv8__main__5mysumB3v12B38c8tJTIeFIjxB2IKSgI4CrvQClQZ6FczSBAA_3dExx =
common local_unnamed_addr global i8* null
@".const.missing Environment:
_ZN08NumbaEnv8__main__5mysumB3v12B38c8tJTIeFIjxB2IKSgI4CrvQClQZ6FczSBAA_3dExx" =
internal constant [99 x i8] c"missing Environment: _ZN08NumbaEnv8__main__5mysumB
3v12B38c8tJTIeFIjxB2IKSgI4CrvQClQZ6FczSBAA_3dExx\00"
@PyExc_RuntimeError = external global i8

; Function Attrs: mustprogress nofree norecurse nosync nounwind willreturn
writeonly
define i32
@_ZN8__main__5mysumB3v12B38c8tJTIeFIjxB2IKSgI4CrvQClQZ6FczSBAA_3dExx(i64*
noalias nocapture writeonly %retptr, { i8*, i32, i8*, i8*, i32 }** noalias
nocapture readnone %excinfo, i64 %arg.a, i64 %arg.b) local_unnamed_addr #0 {
entry:
  %.6 = add nsw i64 %arg.b, %arg.a
  store i64 %.6, i64* %retptr, align 8
  ret i32 0
}

define i8*
@_ZN7cpython8__main__5mysumB3v12B38c8tJTIeFIjxB2IKSgI4CrvQClQZ6FczSBAA_3dExx(i8*
nocapture readnone %py_closure, i8* %py_args, i8* nocapture readnone %py_kws)
local_unnamed_addr {
entry:
  %.5 = alloca i8*, align 8
  %.6 = alloca i8*, align 8
  %.7 = call i32 (i8*, i8*, i64, i64, …) @PyArg_UnpackTuple(i8* %py_args, i8*
getelementptr inbounds ([6 x i8], [6 x i8]* @.const.mysum, i64 0, i64 0), i64 2,
i64 2, i8** nonnull %.5, i8** nonnull %.6)
  %.8 = icmp eq i32 %.7, 0
  %.53 = alloca i64, align 8
  br i1 %.8, label %common.ret, label %entry.endif, !prof !0

common.ret:                                       ; preds =
%entry.endif.endif.endif.endif.endif, %entry.endif.endif.endif, %entry,
%entry.endif.endif.endif.endif.endif.endif, %entry.endif.if
  %common.ret.op = phi i8* [ null, %entry.endif.if ], [ %.74,
%entry.endif.endif.endif.endif.endif.endif ], [ null, %entry ], [ null,
%entry.endif.endif.endif ], [ null, %entry.endif.endif.endif.endif.endif ]
  ret i8* %common.ret.op

entry.endif:                                      ; preds = %entry
  %.12 = load i8*, i8**
@_ZN08NumbaEnv8__main__5mysumB3v12B38c8tJTIeFIjxB2IKSgI4CrvQClQZ6FczSBAA_3dExx,
align 8
  %.17 = icmp eq i8* %.12, null
  br i1 %.17, label %entry.endif.if, label %entry.endif.endif, !prof !0
```

```
entry.endif.if:                                   ; preds = %entry.endif
  call void @PyErr_SetString(i8* nonnull @PyExc_RuntimeError, i8* getelementptr
inbounds ([99 x i8], [99 x i8]* @".const.missing Environment:
_ZN08NumbaEnv8__main__5mysumB3v12B38c8tJTIeFIjxB2IKSgI4CrvQClQZ6FczSBAA_3dExx",
i64 0, i64 0))
  br label %common.ret

entry.endif.endif:                                ; preds = %entry.endif
  %.21 = load i8*, i8** %.5, align 8
  %.24 = call i8* @PyNumber_Long(i8* %.21)
  %.25.not = icmp eq i8* %.24, null
  br i1 %.25.not, label %entry.endif.endif.endif, label %entry.endif.endif.if,
!prof !0

entry.endif.endif.if:                             ; preds = %entry.endif.endif
  %.27 = call i64 @PyLong_AsLongLong(i8* nonnull %.24)
  call void @Py_DecRef(i8* nonnull %.24)
  br label %entry.endif.endif.endif

entry.endif.endif.endif:                          ; preds =
%entry.endif.endif.if, %entry.endif.endif
  %.22.0 = phi i64 [ %.27, %entry.endif.endif.if ], [ 0, %entry.endif.endif ]
  %.32 = call i8* @PyErr_Occurred()
  %.33.not = icmp eq i8* %.32, null
  br i1 %.33.not, label %entry.endif.endif.endif.endif, label %common.ret, !prof
!1

entry.endif.endif.endif.endif:                    ; preds =
%entry.endif.endif.endif
  %.37 = load i8*, i8** %.6, align 8
  %.40 = call i8* @PyNumber_Long(i8* %.37)
  %.41.not = icmp eq i8* %.40, null
  br i1 %.41.not, label %entry.endif.endif.endif.endif.endif, label
%entry.endif.endif.endif.endif.if, !prof !0

entry.endif.endif.endif.endif.if:                 ; preds =
%entry.endif.endif.endif.endif
  %.43 = call i64 @PyLong_AsLongLong(i8* nonnull %.40)
  call void @Py_DecRef(i8* nonnull %.40)
  br label %entry.endif.endif.endif.endif.endif

entry.endif.endif.endif.endif.endif:              ; preds =
%entry.endif.endif.endif.endif.if, %entry.endif.endif.endif.endif
  %.38.0 = phi i64 [ %.43, %entry.endif.endif.endif.endif.if ], [ 0,
%entry.endif.endif.endif.endif ]
  %.48 = call i8* @PyErr_Occurred()
  %.49.not = icmp eq i8* %.48, null
```

```
  br i1 %.49.not, label %entry.endif.endif.endif.endif.endif.endif, label
%common.ret, !prof !1

entry.endif.endif.endif.endif.endif.endif:        ; preds =
%entry.endif.endif.endif.endif.endif
  store i64 0, i64* %.53, align 8
  %.57 = call i32
@_ZN8__main__5mysumB3v12B38c8tJTIeFIjxB2IKSgI4CrvQClQZ6FczSBAA_3dExx(i64*
nonnull %.53, { i8*, i32, i8*, i8*, i32 }** nonnull undef, i64 %.22.0, i64
%.38.0) #1
  %.67 = load i64, i64* %.53, align 8
  %.74 = call i8* @PyLong_FromLongLong(i64 %.67)
  br label %common.ret
}

declare i32 @PyArg_UnpackTuple(i8*, i8*, i64, i64, …) local_unnamed_addr

declare void @PyErr_SetString(i8*, i8*) local_unnamed_addr

declare i8* @PyNumber_Long(i8*) local_unnamed_addr

declare i64 @PyLong_AsLongLong(i8*) local_unnamed_addr

declare void @Py_DecRef(i8*) local_unnamed_addr

declare i8* @PyErr_Occurred() local_unnamed_addr

declare i8* @PyLong_FromLongLong(i64) local_unnamed_addr

; Function Attrs: mustprogress nofree norecurse nosync nounwind willreturn
writeonly
define i64
@cfunc._ZN8__main__5mysumB3v12B38c8tJTIeFIjxB2IKSgI4CrvQClQZ6FczSBAA_3dExx(i64
%.1, i64 %.2) local_unnamed_addr #0 {
entry:
  %.4 = alloca i64, align 8
  store i64 0, i64* %.4, align 8
  %.8 = call i32
@_ZN8__main__5mysumB3v12B38c8tJTIeFIjxB2IKSgI4CrvQClQZ6FczSBAA_3dExx(i64*
nonnull %.4, { i8*, i32, i8*, i8*, i32 }** nonnull undef, i64 %.1, i64 %.2) #1
  %.18 = load i64, i64* %.4, align 8
  ret i64 %.18
}

attributes #0 = { mustprogress nofree norecurse nosync nounwind willreturn
writeonly }
attributes #1 = { noinline }
```

```
!0 = !{!"branch_weights", i32 1, i32 99}
!1 = !{!"branch_weights", i32 99, i32 1}
```

1.8 Numba in general

- Numba has a number of features to not only target CPUs, but also GPU architectures.
- It is a fast moving project and widely used to speed up Python code.
- Achieve close to C and FORTRAN speeds!
- Tell the Python haters to go away!

[ ]: